COURSEHIVE

E-Learning Platform with Learning Analytics

Project Documentation

Python | Flask | MySQL | HTML | CSS | JavaScript | Pandas | NumPy | Matplotlib

1. Abstract

CourseHive is a web-based e-learning platform built with Python and Flask, with MySQL used to store application data. It brings course and enrollment workflows together with lessons, progress tracking, quizzes, assignments, reviews, and study-session tracking. An analytics area summarizes recorded platform activity through reports and charts generated with Python data tools.

CourseHive analytics are descriptive. They summarize activity recorded by the application and should not be interpreted as proof of a learner's ability, motivation, or reasons for inactivity.

2. Problem Statement

Course and learning activity records can be difficult to review when enrollment, lesson completion, assessment, and study-session information are spread across different records. CourseHive provides a unified web application for learning workflows and summarizes selected records to help learners review progress and administrators understand recorded platform activity.

3. Objectives

- Manage courses and learner enrollments.
- Track lesson completion and timestamps.
- Support quizzes, assignments, reviews, and study sessions.
- Provide separate workflows for students, instructors, and administrators.
- Summarize enrollment, completion, progress, and study-time information.
- Display selected analytics as charts for easier interpretation.

4. Scope

Included features

- Course and enrollment management.
- Lesson progress tracking and completion timestamps.
- Quiz attempts, assignment submissions, reviews, and ratings.
- Study-session start/stop tracking and study-time reports.
- Administrator analytics and visual reports.

Interpretation limits

- No recorded activity does not prove that a learner stopped studying; learning may happen outside the platform.
- The inactive-enrollment report uses a 14-day cutoff in the current project context. Confirm the exact query and definition in the current code.
- Verify each feature and report against the current project before formal submission.

5. Technologies Used

Technology	Purpose	Use in CourseHive
Python	Programming language	Application logic and data processing

Flask	Web framework	Routes, requests, sessions, and templates
MySQL	Relational database	Stores learners, courses, enrollments, and activity
HTML / CSS / JavaScript	Frontend	Page structure, styling, and browser interactions
mysql-connector-python	Database connector	Connects Python to MySQL
python-dotenv	Configuration	Loads environment variables
Pandas / NumPy	Data processing	Supports report calculations and aggregation
Matplotlib	Visualization	Creates charts for analytics pages
Git / GitHub	Version control	Tracks and stores project source code

6. System Architecture

CourseHive follows a layered web application design:

- Presentation layer: HTML templates, CSS, and JavaScript in the browser.
- Application layer: Flask routes process requests and apply application logic.
- Data layer: MySQL stores relational application records.
- Analytics layer: Python processes selected data; Pandas and NumPy support calculations, while Matplotlib creates charts.

Typical flow: Browser → Flask route → MySQL query/update → Python processing when needed → HTML report or chart → Browser.

7. Main Modules

7.1 User and role workflows

CourseHive provides student, instructor, and administrator workflows. Access to administrative analytics should be restricted to authorized administrators. Verify the actual route guards and permissions before production release.

7.2 Courses and enrollments

Course records represent learning offerings, while enrollment records associate learners with courses and store enrollment-related information.

7.3 Lessons and progress

Lessons belong to courses. Lesson progress records store whether a learner completed a lesson and may store the completion timestamp. The student dashboard's configured weighted progress model is lessons 50%, assignments 25%, and quizzes 25%.

7.4 Quizzes and assignments

The database includes quizzes, quiz questions, quiz attempts, assignments, and assignment submissions. These records support assessment-related workflows.

7.5 Study sessions

Study-session records capture a learner, course, start/end times, and duration-related fields. Reports reflect only sessions recorded by CourseHive, not every minute a learner may spend studying.

7.6 Reviews and ratings

Course reviews and ratings provide learner feedback that can be summarized in course-level reports.

7.7 Analytics and charts

The project context includes course/enrollment, student, completion/progress, study-time, weekly activity, and visual reports. Matplotlib charts are generated and embedded in pages as Base64-encoded images. Check the current code before claiming that every report is complete or tested.

8. Analytics Definitions

- Enrollment count: number of enrollment records matching the report's course or time-period filters.
- Lesson completion: based on recorded lesson_progress values and completion timestamps.
- Overall progress: configured weighting is lessons 50%, assignments 25%, quizzes 25%.
- Certificate eligibility: configured for Completed status and progress of at least 100%.
- Inactive enrollment: current project context uses a 14-day cutoff and recorded activity logic; confirm the precise query in the current implementation.
- Weekly activity: project context indicates a report covering the last 12 weeks.
- Missing activity can reflect off-platform learning or incomplete tracking and is not definitive evidence of disengagement.

9. Database Design

The CourseHive MySQL database includes the following tables in the project context:

- students — learner profiles and account-related records.
- instructors — instructor records.
- admins — administrator accounts.
- courses — course information.
- course_instructors — association between courses and instructors.
- enrollments — learner-to-course enrollment records and status.
- lessons and lesson_progress — course lessons and learner completion records.
- quizzes and quiz_questions — quiz definitions and questions.
- quiz_attempts — learner quiz attempts/results.
- assignments and assignment_submissions — assignment details and submissions.
- study_sessions — recorded study activity and duration.
- reviews — course reviews and ratings.
- course_requests — course-related requests.

Review primary keys, foreign keys, indexes, and constraints in the current SQL schema. The project context indicates that lesson_progress has a unique student/lesson pair to avoid duplicate progress rows for the same lesson.

10. Setup and Execution

Prerequisites

- Python installed.
- MySQL Server or an accessible MySQL-compatible database.
- A terminal such as Windows Command Prompt or PowerShell.
- Git, if cloning the repository.

Local setup steps

1. Open the CourseHive project folder in a terminal.
2. Create a virtual environment: python -m venv .venv
3. Activate it in Windows Command Prompt: .venv\Scripts\activate.bat
4. Install dependencies: pip install -r requirements.txt
5. Create the database and import the project's schema/backup if available.
6. Set the database connection values in .env; never commit real credentials.
7. Confirm db.py uses the correct environment variable names and database settings.
8. Run the application with: python app.py
9. Open the local address printed in the terminal (commonly http://127.0.0.1:5000).

The exact environment variable names and database import command depend on the current .env, db.py, and SQL backup. Use the values configured for your own environment.

11. Deployment Notes

- Keep passwords and secret keys in environment variables rather than source code.
- For hosted MySQL services, use the provider's host, port, database, username, password, and required SSL settings.
- For a hosted Flask service, install requirements and configure a production WSGI server/start command.
- A deployment configuration or GitHub repository does not by itself confirm that the live application is working.
- Verify the deployment URL, database connectivity, and role permissions before sharing the application publicly.

12. Testing Checklist

- Verify login/logout and role-based access for student, instructor, and admin accounts.
- Create a course and verify enrollment records.
- Complete a lesson and verify the stored completion flag and timestamp.
- Submit a quiz attempt and assignment submission with valid test data.
- Start and stop a study session; verify timestamps and duration.
- Check the weighted progress calculation: lessons 50%, assignments 25%, quizzes 25%.
- Confirm certificate access requires Completed status and at least 100% progress.
- Compare analytics report totals with known database records.
- Test chart labels, empty-data cases, date ranges, and access restrictions.
- Test the inactive-report cutoff around the 14-day boundary.
- Test missing environment variables and database connection failures.

13. Limitations

- Analytics depend on the completeness and correctness of stored records.
- Study-time reports include only sessions tracked by the application.
- The system cannot account for all learning done outside CourseHive.
- The inactivity definition and completion-rate denominator should be displayed clearly.
- Production deployment, permissions, and report calculations require final verification.

14. Future Enhancements

- Add automated unit and integration tests.
- Allow analytics reports to be exported to CSV or PDF.
- Add configurable date ranges and clearer metric definitions.
- Improve chart accessibility with text summaries and descriptive labels.
- Optimize queries and add pagination for larger datasets.
- Add privacy-conscious data retention and production monitoring.

15. Conclusion

CourseHive combines e-learning workflows with descriptive learning analytics in a Flask/MySQL application. Course management, progress tracking, study-session records, and administrator reports demonstrate the integration of web development, relational databases, data processing, visualization, and version control. Before formal submission or public deployment, compare this document with the current code and test each feature.

Appendix: Simplified Project Structure

CourseHive

```
app.py                  # Flask application and routes
db.py                   # MySQL connection helpers
create_admin.py         # Admin creation utility
requirements.txt        # Python dependencies
.env                    # Local environment settings (not committed)
templates/              # HTML/Jinja templates
static/css/style.css    # Stylesheet
```